



서울시립대학교

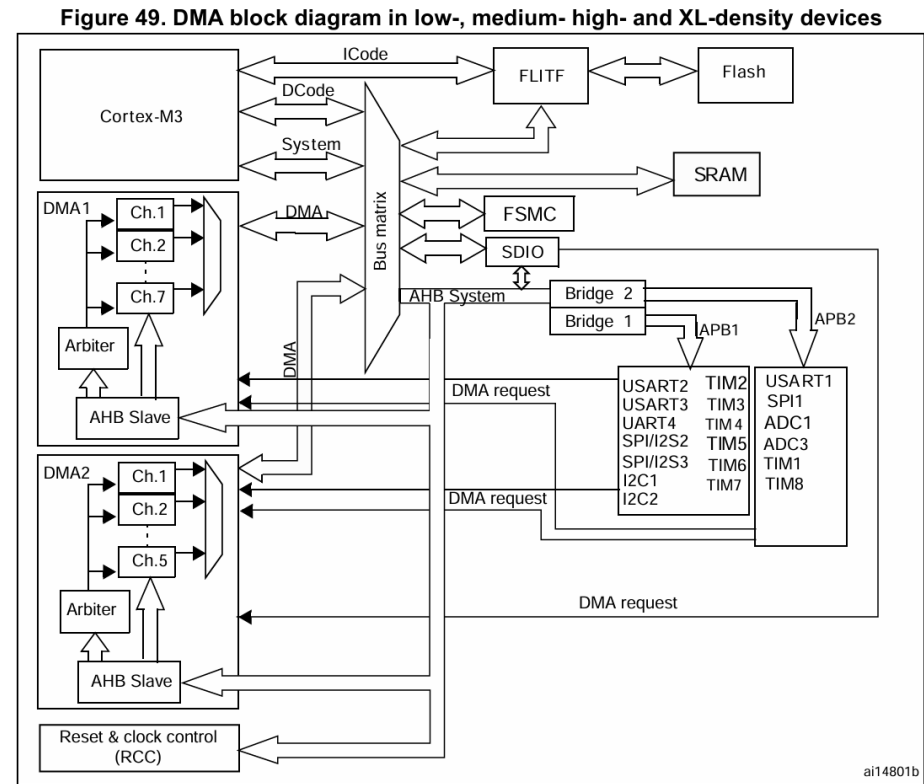
Lec 8 - DMA

전력전자연구실 교수 이 승 환



DMA(Direct Memory Access)

- MCU 내부 memory와 MCU에 연결되는 주변 장치들 사이에 CPU 간섭 없이 데이터를 전송하게 해줌
- UART, ADC, DAC, SPI 등 peripheral 에서 발생하는 대량의 데이터 전송을 CPU가 일일이 진행하지 않고, DMA가 처리해서 데이터 access 빠름
- 내부 memory 사이의 데이터 통신에도 사용 가능.
- DMA 동작은 DMA controller에 의해서 수행.
- 기본적으로 4개의 register들로 구성되고, TCB(Transfer Control Block)이라는 구조체로 정의하여 사용.
 - 전송될 데이터의 크기를 포함하는 counter register,
 - Source address register,
 - Destination address register,
 - Status register



ai14801b

STM32 DMA 소개

- STM32F103RB는 2개의 DMA controller를 가짐.
 - DMA1은 7개 channels를 가짐.
 - DMA2는 5개 channels를 가짐.

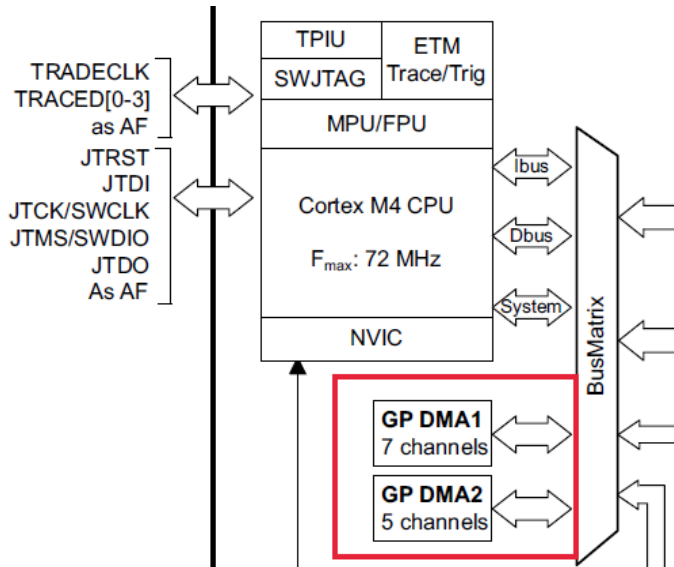
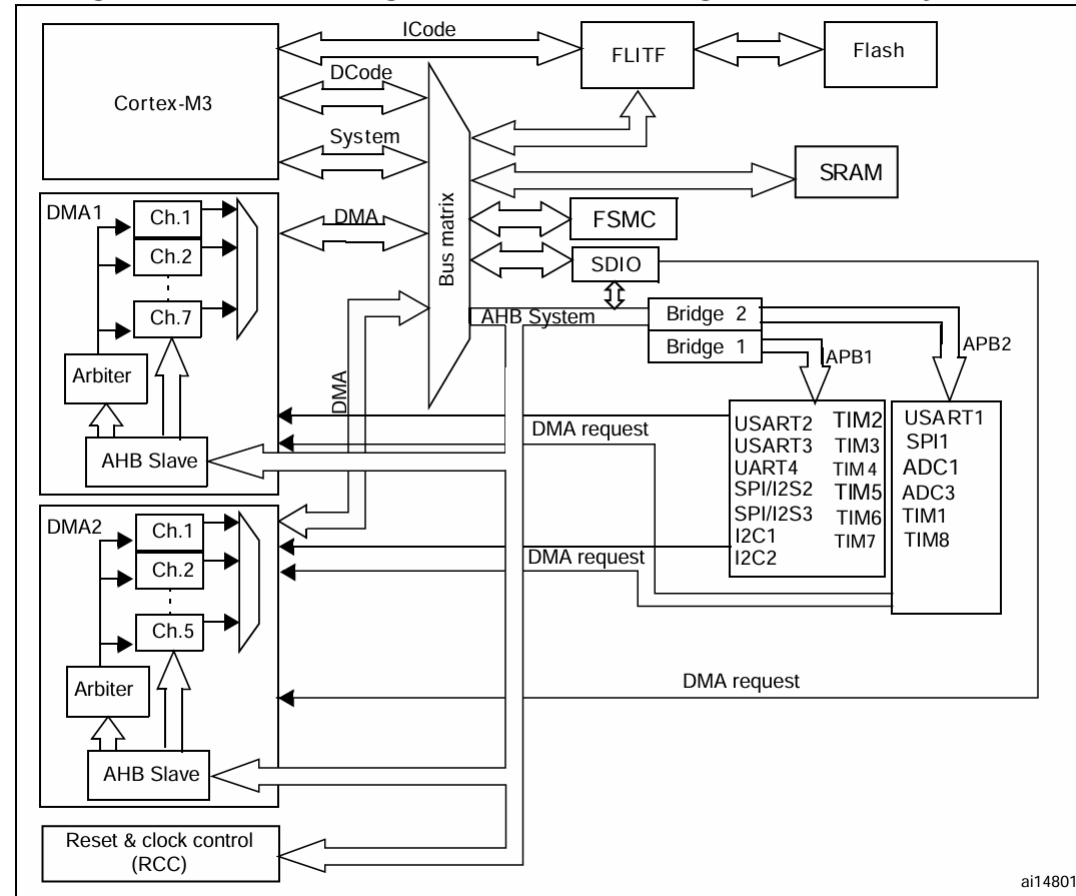


Figure 49. DMA block diagram in low-, medium- high- and XL-density devices



ai14801b

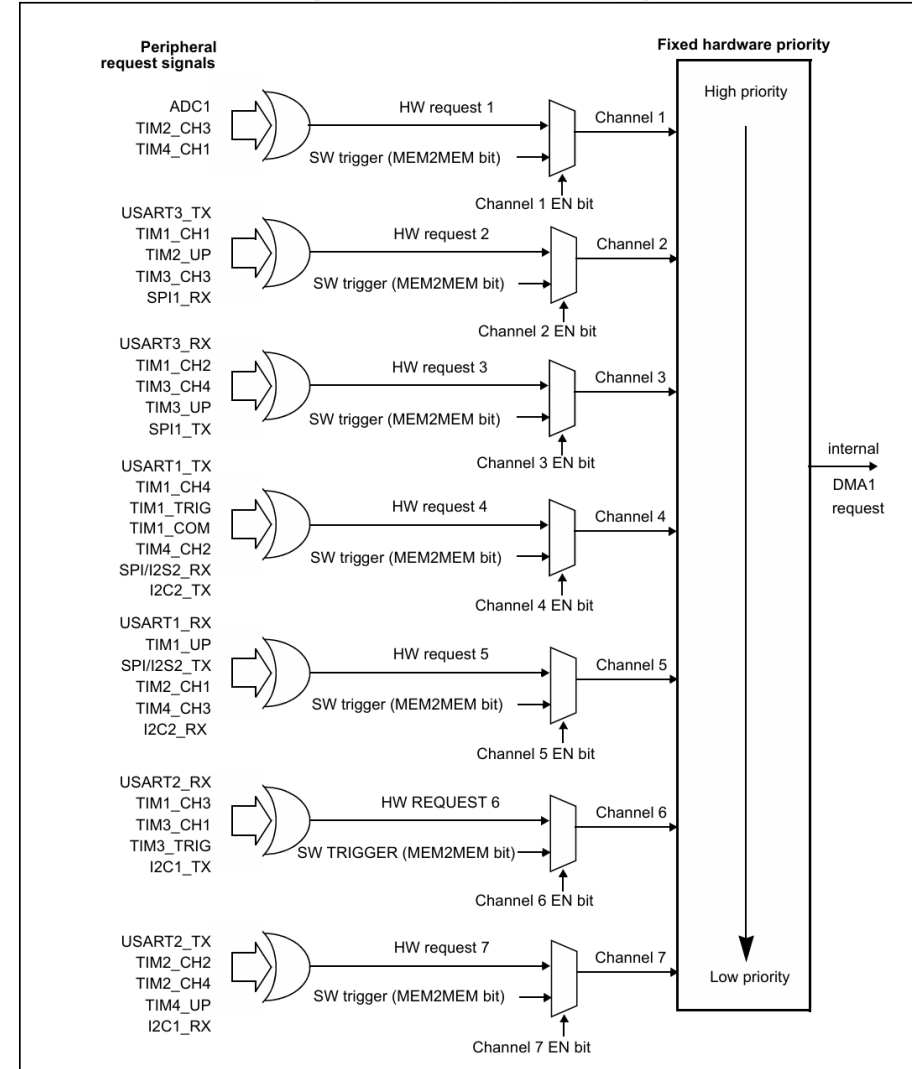
STM32 DMA controller 특징.

- 각각의 channel 은 하나 또는 그 이상의 주변 장치(peripheral)들로부터 memory 접근 요청 수행 가능
- 각각의 channel 은 전용 peripheral 에 하드웨어적으로 연결되어 있어 (hardware DMA request line), 이것을 통하여 DMA 동작 요청.
- Software trigger 기능도 제공.
- DMA 요청의 우선 순위도 지원 - very high, high, medium, low 이렇게 4개의 level 설정 가능.
- 하나의 DMA controller는 DMA 요청들 사이에 우선순위를 관리하기 위한 중재자(arbiter)를 사용.
- 모든 DMA controller는 각각 DMA bus를 통하여 bus matrix에 연결.

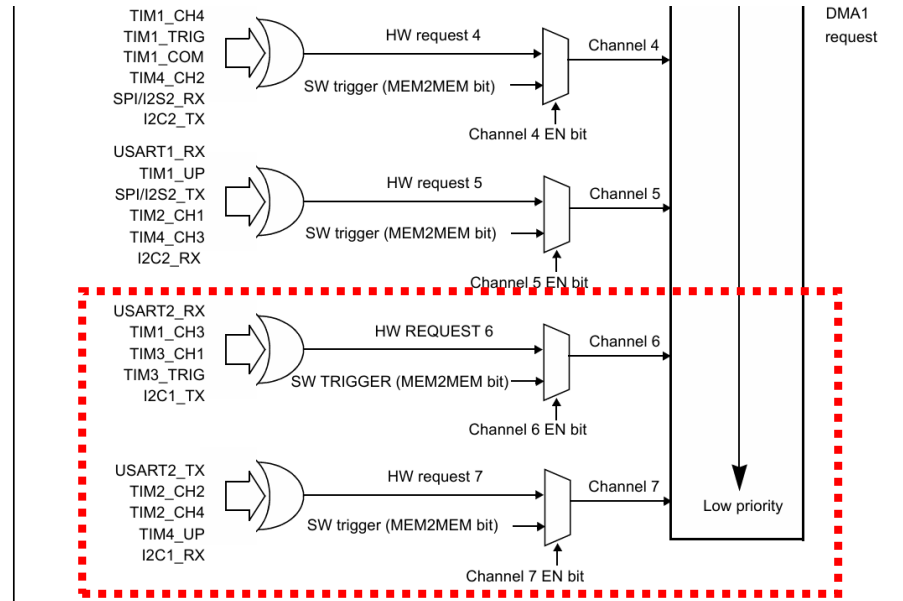
DMA 전송 방식의 종류 :

- Block 전송 : 가장 많이 사용하는 방식, 한번 요청으로 지정된 메모리 블록 전체 전송, CPU 개입 거의 없어 효율적
- 그 밖에 Cycle Stealing, Interleaved DMA 전송 방식: CPU와 버스 번갈아가면서 사용, 실시간 응답성이 필요한 경우 유용

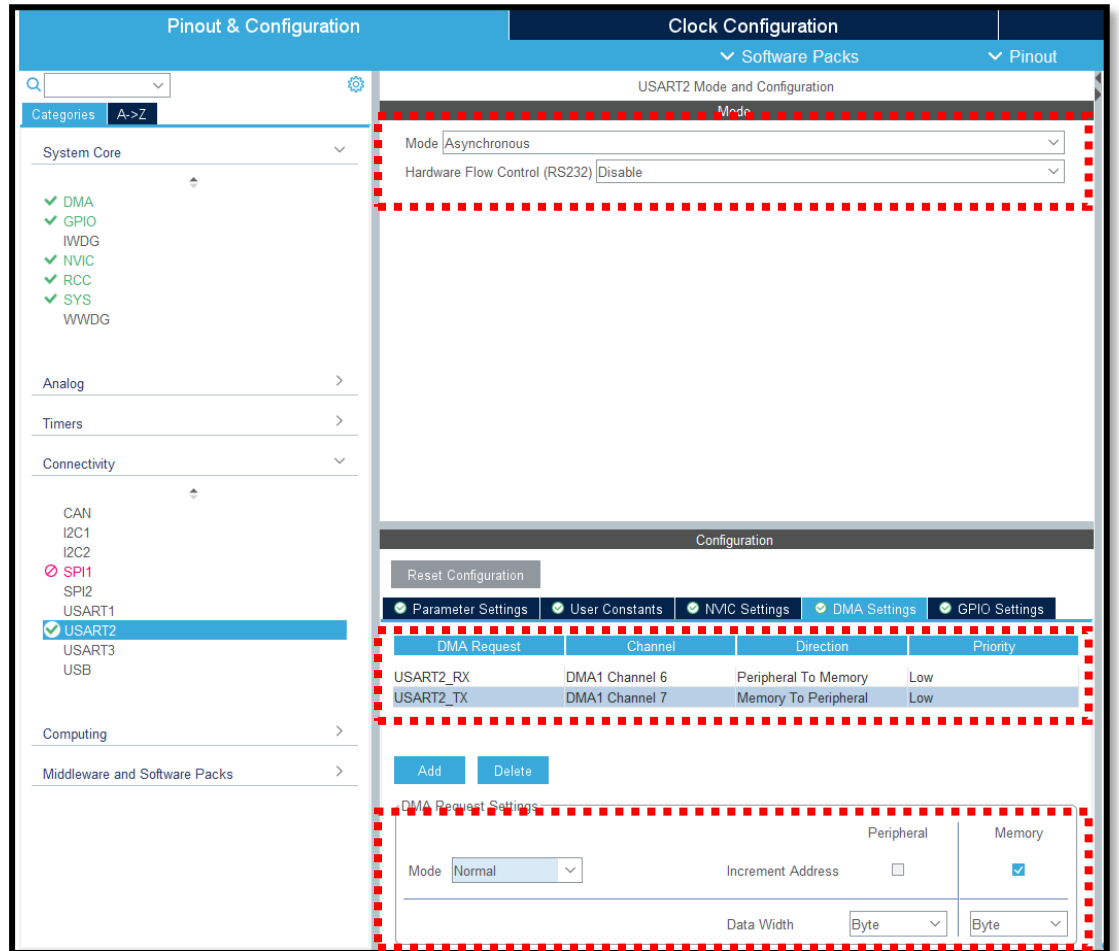
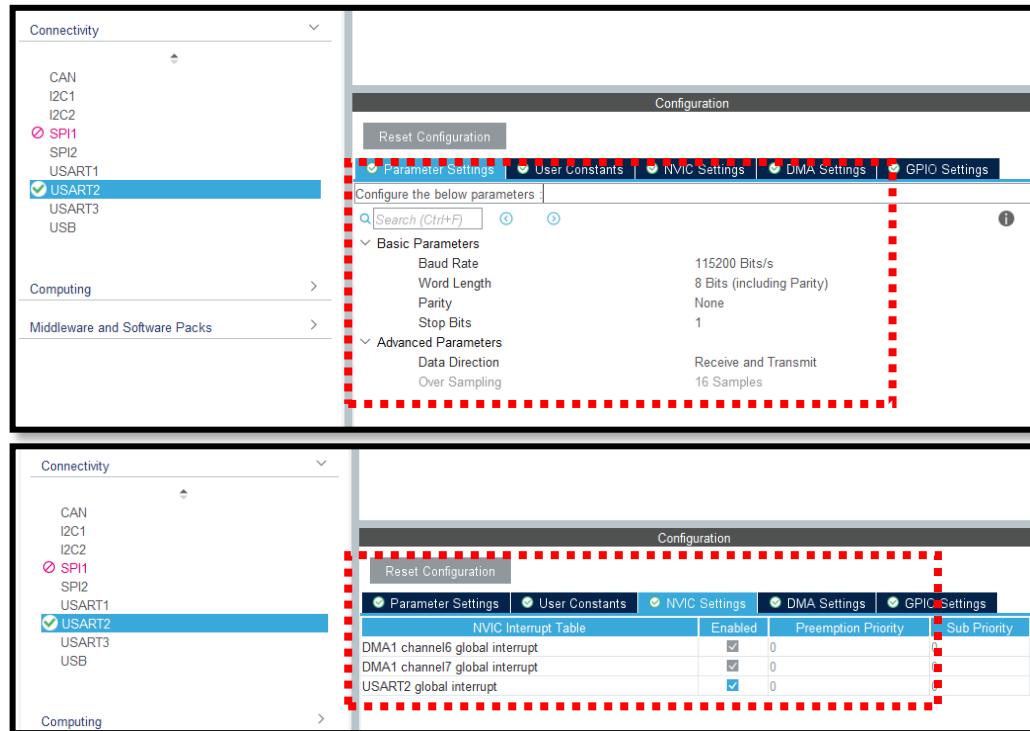
Figure 50. DMA1 request mapping



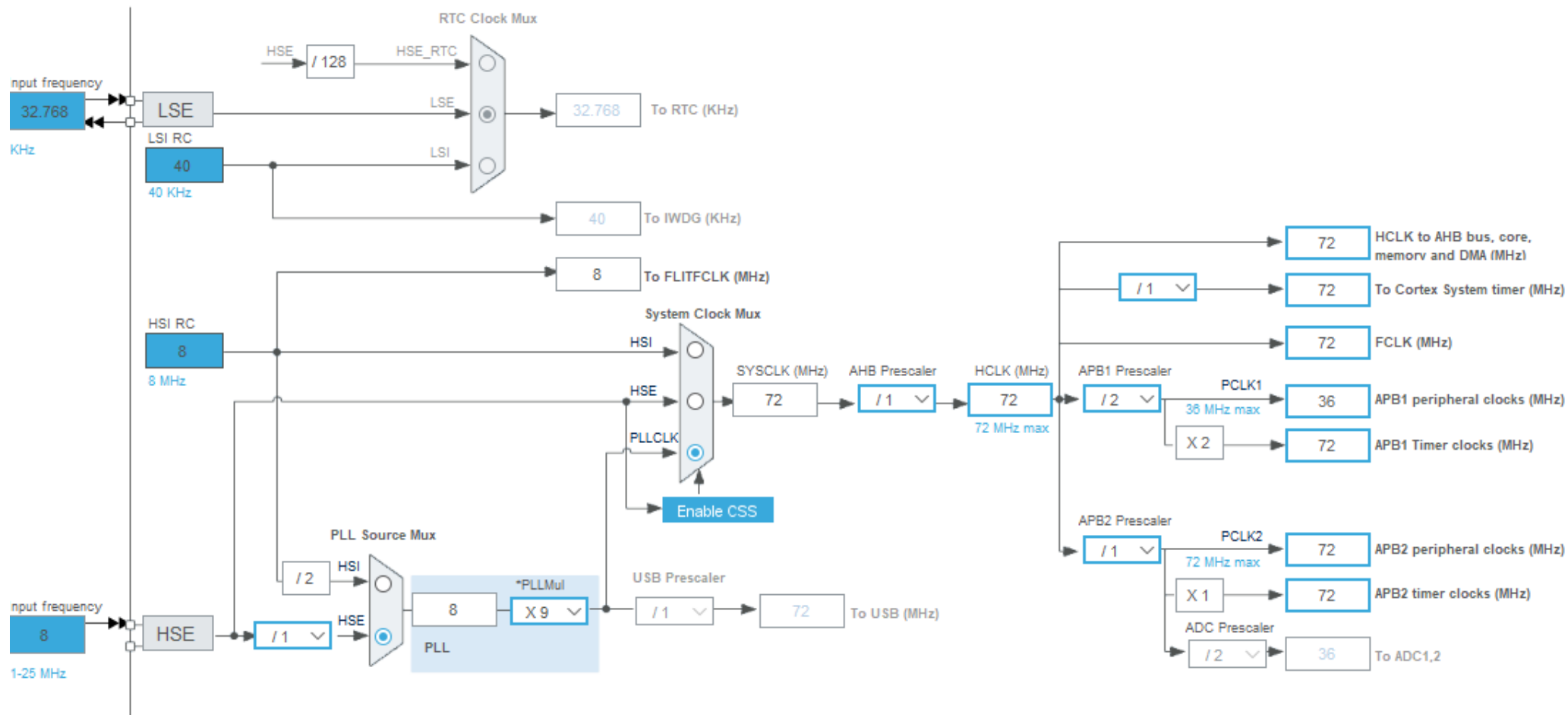
- ✓ 실습 목표: Tera Term에서 임의의 길이 문자열을 파일로 보내면 DMA 이용해서 메모리에 저장했다가, 데이터 입력이 끝난 IDLE 시점에 받은 문자열 그대로 다시 MCU에서 PC로 보내는 프로그램
 - USART2는 DMA1 request channel 중 6과 7에 연결되어 있음.
 - Channel 번호가 낮을수록 high priority



✓ USART2 Configuration



✓ Clock 설정



✓ UART DMA 사용 할 때에 주의사항

- HAL 함수인 HAL_UART_Receive_DMA(&huart2, UartRxDMA_Buf, RXBUF_MAX); // RXBUF_MAX = 303 사용
- 위 HAL 함수 사용하는 경우 지정한 수신 또는 송신할 데이터 (RXBUF_MAX = 303 byte) 가 모두 채워 질 때까지 interrupt가 발생하지 않음.
- 303개의 문자를 수신할 때까지 DMA interrupt 발생하지 않음! 따라서 데이터가 max buffer size 까지 필요하지 않은 경우 문제 발생
- **UART_IDLE interrupt**를 사용하여 문제 해결: 일정 시간 동안 데이터가 수신되지 않을 때 발생하는 인터럽트
- 이것 이용하면 DMA 버퍼가 다 차지 않아도 데이터가 더 이상 안들어오면 MCU가 IDLE interrupt 받아서 데이터 처리 가능
- UART_IDLE interrupt 사용 방법 :
 - UART_IDLE interrupt가 발생하면, SR.IDLE flag = 1로 셋팅됨
 - huart2.Instance->SR 통해서 USART status register read 하고 huart2.Instance->DR 통해서 data register read 하면 SR.IDLE =0 으로 클리어 가능 (MCU reference manual 참고)
 - 이후 수동으로 DMA를 disable하고, (HAL_UART_DMASStop(&huart2))
 - DMA에 남은 데이터 개수 확인한 후
 - 수신된 데이터 길이 계산 및
 - 버퍼 처리한 후 DMA 재시작!!

✓ main.c 의 수정

```
/* Private includes -----
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */
```

```
/* USER CODE BEGIN PV */
uint8_t UartRxDMA_Buf[RXBUF_MAX];
volatile uint16_t UartRxCount = 0;
/* USER CODE END PV */
```

```
/* USER CODE BEGIN 0 */
static void UART2_DMA_Ready(void){
    /* IDLE interrupt enable */
    __HAL_UART_ENABLE_IT(&huart2, UART_IT_IDLE);

    /* RX DMA start */
    HAL_UART_Receive_DMA(&huart2, UartRxDMA_Buf, RXBUF_MAX);
}
```

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_USART2_UART_Init();
/* USER CODE BEGIN 2 */
UART2_DMA_Ready();
/* USER CODE END 2 */
```

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */
```

```
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        | }
    }
}
```

✓ stm32f1xx_it.c의 수정

```
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN EV */
extern uint8_t UartRxDMA_Buf[]; // main.c에 정의
extern volatile uint16_t UartRxCount;
/* USER CODE END EV */
```

✓ main.h의 수정

```
/* USER CODE BEGIN Private defines */
#define RXBUF_MAX 256
/* USER CODE END Private defines */
```

```
void USART2_IRQHandler(void)
{
    /* USER CODE BEGIN USART2_IRQn 0 */
    if (__HAL_UART_GET_FLAG(&huart2, UART_FLAG_IDLE)) {
        volatile uint32_t tmp;
        tmp = huart2.Instance->SR; // SR read
        tmp = huart2.Instance->DR; // DR read
        (void)tmp;

        HAL_UART_DMASstop(&huart2); // DMA 강제 종료

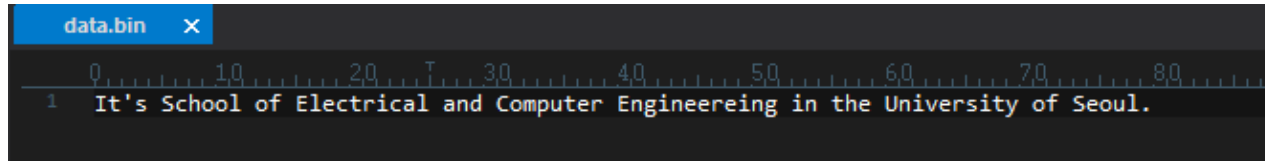
        /* 남은 카운터로 실제 수신 길이 계산 */
        uint16_t remaining = __HAL_DMA_GET_COUNTER(huart2.hdmarx);
        UartRxCount = RXBUF_MAX - remaining;

        /* 에코(수신한 만큼만 전송) */
        if (UartRxCount > 0)
            HAL_UART_Transmit(&huart2, UartRxDMA_Buf, UartRxCount, 100);

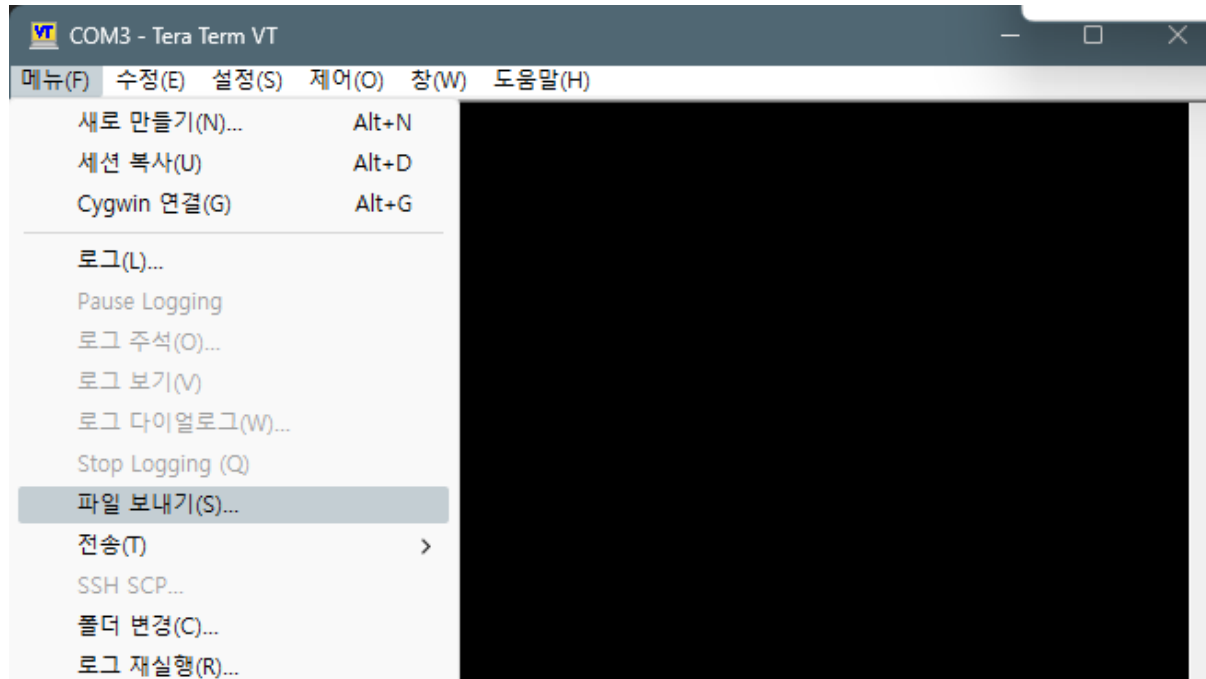
        /* 버퍼 정리 및 재시작 */
        memset(UartRxDMA_Buf, 0, RXBUF_MAX);
        HAL_UART_Receive_DMA(&huart2, UartRxDMA_Buf, RXBUF_MAX);
    }
    /* USER CODE END USART2_IRQn 0 */
    HAL_UART_IRQHandler(&huart2);
    /* USER CODE BEGIN USART2_IRQn 1 */

    /* USER CODE END USART2_IRQn 1 */
}
```

- ✓ 메모장 이용해 임의의 텍스트 작성후 data.bin으로 저장



- ✓ TeraTerm 통해 data.bin 파일 전송 후 터미널 창 및 Live Watch 창 확인



USART2 이용한 DMA 실습

- ✓ 오른쪽 그림과 같이 breakpoint 설정 후 실행해보면
 - idle 상태일 때 인터럽트 발생시켜 IRQhandler 에 진입한 것 알 수 있고,
 - UartRxDMA_Buf 에 텍스트가 전부 들어와있는 것 확인 가능
 - 이후 실행하면 다시 TeraTerm에 수신한 문자를 보내고 UartRxDMA_Buf clear 한 후 다시 USART2_DMA receive 상태로 대기
 - 데이터 수신 loss 가 있는지 없는지 여러 번 긴 문장을 이용해 확인

The screenshot shows the IAR Embedded Workbench IDE with the following components:

- Workspace:** Displays the project structure for 'UART_DMA_Ex', including files like 'main.c', 'stm32f1xx_hal_dma.c', and 'stm32f1xx_it.c'.
- Code Editor:** Shows the 'USART2_IRQHandler' function in 'main.c'. The code includes comments in Korean and C code for handling USART2 interrupts and DMA. A breakpoint is set at line 253, marked with a red dot.
- Live Watch:** A table showing the state of 'UartRxDMA_Buf' and other variables at the breakpoint.

Expression	Value	Location
UartRxDMA_Buf	<array>"It...	0x2000'0058
[0]	'I' (0x49)	0x2000'0058
[1]	't' (0x74)	0x2000'0059
[2]	' ' (0x20)	0x2000'005a
[3]	's' (0x73)	0x2000'005b
[4]	' ' (0x20)	0x2000'005c
[5]	'S' (0x53)	0x2000'005d
[6]	'c' (0x63)	0x2000'005e
[7]	'h' (0x68)	0x2000'005f
[8]	'o' (0x6F)	0x2000'0060
[9]	'o' (0x6F)	0x2000'0061
[10]	'l' (0x6C)	0x2000'0062
[11]	' ' (0x20)	0x2000'0063
[12]	'o' (0x6F)	0x2000'0064
[13]	'f' (0x66)	0x2000'0065
[14]	' ' (0x20)	0x2000'0066
[15]	'E' (0x45)	0x2000'0067
[16]	'l' (0x6C)	0x2000'0068
[17]	'e' (0x65)	0x2000'0069
[18]	'c' (0x63)	0x2000'006a
[19]	't' (0x74)	0x2000'006b
[20]	'r' (0x72)	0x2000'006c
[21]	'i' (0x69)	0x2000'006d
[22]	'c' (0x63)	0x2000'006e
[23]	'a' (0x61)	0x2000'006f
[24]	'l' (0x6C)	0x2000'0070
[25]	' ' (0x20)	0x2000'0071
[26]	'a' (0x61)	0x2000'0072
[27]	'n' (0x6E)	0x2000'0073
[28]	'd' (0x64)	0x2000'0074
[29]	' ' (0x20)	0x2000'0075
[30]	'c' (0x43)	0x2000'0076
[31]	'o' (0x6F)	0x2000'0077
[32]	'm' (0x6D)	0x2000'0078
[33]	'p' (0x70)	0x2000'0079
[34]	'u' (0x75)	0x2000'007a
[35]	't' (0x74)	0x2000'007b
[36]	'e' (0x65)	0x2000'007c
[37]	'r' (0x72)	0x2000'007d
[38]	' ' (0x20)	0x2000'007e
[39]	'E' (0x45)	0x2000'007f
[40]	'n' (0x6E)	0x2000'0080
[41]	'g' (0x67)	0x2000'0081
[42]	'i' (0x69)	0x2000'0082
[43]	'n' (0x6E)	0x2000'0083
[44]	'e' (0x65)	0x2000'0084
[45]	'e' (0x65)	0x2000'0085
[46]	'r' (0x72)	0x2000'0086
[47]	'e' (0x65)	0x2000'0087
[48]	'i' (0x69)	0x2000'0088
[49]	'n' (0x6E)	0x2000'0089
[50]	'a' (0x61)	0x2000'008a
- Debug Log:** Shows the execution log, including a breakpoint hit at 'stm32f1xx_it.c:253.7'.

- ✓ 해당 구현 결과에 대해 같은 data를 계속 보내면서 데이터 수신에 정확성 검토해보면 data 수신이 완벽히 되지 않는 것 알 수 있음.
- ✓ 데이터 손실 원인
 - 데이터 처리를 위해 DMA 를 stop 하면 그동안 UART 에서 보내는 데이터 유실 (Overrrun Error, ORE)
 - HAL_UART_Transmit 의 timeout 이 너무 짧으면 데이터 수신은 잘됐으나 TeraTerm에 보이는 것만 잘려서 보일 수도 있음(이 부분은 메모리 확인 필요)
- ✓ 따라서 DMA를 stop 하면 안되고, 처리하는 중에도 buffer에 계속 데이터 받아야 함.
 - 이를 위해서는 데이터 쌓는 buffer에 다 차면 다시 index를 0으로 돌려서 거기서 부터 받을 수 있게 해줘야 함
 - DMA circular mode는 buffer 끝까지 다 차면 다시 0번지로 돌아가는 모드임.
 - DMA normal mode 는 정해진 길이만큼 buffer 다 DMA complete IT 발생시키고 끝임. 이 경우에는 DMA 수동 재시작 필요함.
 - DMA circular mode 사용하는 경우 소프트웨어 상에서 DMA 의 데이터를 읽는 것도 이에 맞춰서 읽어야 함.

✓ Ring buffer

- 총 8칸짜리 버퍼가 있으면

Index: 0 1 2 3 4 5 6 7

Buffer: [] [] [] [] [] [] [] []

- 데이터가 들어오면 index 0 부터 차근차근 채워짐.
- 이번에 들어온 데이터에 대한 시작과 끝에 대한 index만 가지고 있으면 됨.
- head: 이번에 들어온 데이터의 끝 index
- tail: 이번에 들어온 데이터의 시작 index
- 만약 빈 버퍼에 총 4바이트가 들어왔다면 다음과 같이 head 와 tail index 만들어주면 됨.

Index: 0 1 2 3 4 5 6 7

Buffer: [] [] [] [] [] [] [] []

↑ ↑
tail head

- ✓ stm32f1xxx_it.c 의 USART2_IRQHandler 에서는 SR 과 DR read 만 하고 main 에서 처리할 수 있도록 RxDMA_Data_Flag =1 로 셋팅

```

/* USER CODE BEGIN EV */
extern uint8_t UartRxDMA_Buf[]; // main.c 222 2222
extern volatile uint16_t tail;
extern volatile uint16_t head;
extern volatile uint8_t RxDMA_Data_Flag;

/* USER CODE END EV */

void USART2_IRQHandler(void)
{
    /* USER CODE BEGIN USART2_IRQn 0 */
    if (__HAL_UART_GET_FLAG(&huart2, UART_FLAG_IDLE)) {
        volatile uint32_t tmp;
        tmp = huart2.Instance->SR; // SR read
        tmp = huart2.Instance->DR; // DR read
        (void)tmp;

        RxDMA_Data_Flag = 1;
    }
    /* USER CODE END USART2_IRQn 0 */
    HAL_UART_IRQHandler(&huart2);
    /* USER CODE BEGIN USART2_IRQn 1 */

    /* USER CODE END USART2_IRQn 1 */
}

```

✓ main 에서는 다음과 같이 수정

- index MAX 에서 현재 채워진 counter 값을 빼서 이번에 들어온 데이터의 최후의 값으로 head 값 설정
- RxDMA_Data_Flag = 0으로 클리어하고,
- head > tail 이면 계속 해서 transmit 하고
- 만약 head < tail 이면 index 가 wrap around 한 것이라서 오른쪽과 같이 transmit 해주면 됨.
- 이게 바로 ring buffer 처리 해주는 것
- 데이터 transmit 끝나면 tail = head로 업데이트 해주고, while 문으로 계속 돌리면서 head = tail 이 될 때까지 계속 전송해줌
- 이렇게 하면 수신에도 누락이 없고 송신에도 누락이 없음.

```
/* USER CODE BEGIN PV */
uint8_t UartRxDMA_Buf[RXBUF_MAX];
volatile uint16_t tail = 0;
volatile uint16_t head = 0;
volatile uint8_t RxDMA_Data_Flag = 0;
```

```
/* Infinite Loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    if(RxDMA_Data_Flag == 1){
        __disable_irq();
        head = RXBUF_MAX - __HAL_DMA_GET_COUNTER(huart2.hdmarx);
        RxDMA_Data_Flag = 0;
        __enable_irq();

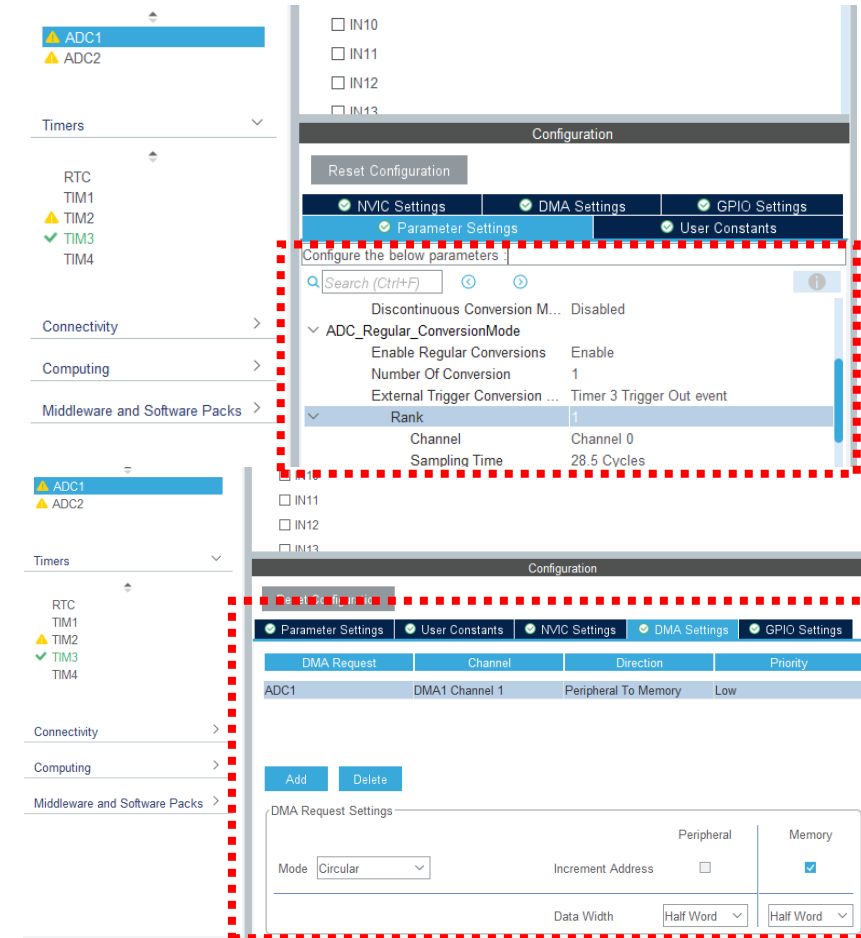
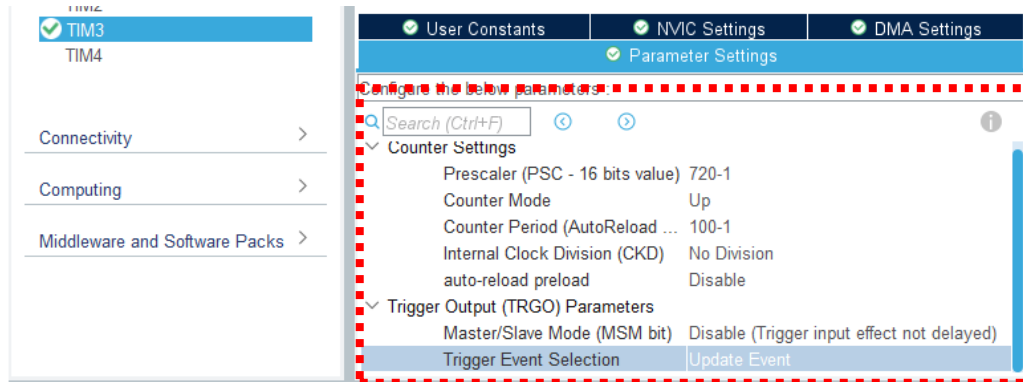
        while(head != tail){
            if(head > tail){
                HAL_UART_Transmit(&huart2, &UartRxDMA_Buf[tail], head-tail, 1000);
            }else{
                HAL_UART_Transmit(&huart2, &UartRxDMA_Buf[tail], RXBUF_MAX-tail, 1000);
                HAL_UART_Transmit(&huart2, &UartRxDMA_Buf[0], head, 1000);
            }
            tail = head;
            head = RXBUF_MAX - __HAL_DMA_GET_COUNTER(huart2.hdmarx);
        }
    }
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
```


✓ 실습 목표: TIM3 (1kHz) → ADC1 트리거 → DMA Circular 100 sample buffer 저장 → 100개 채워질 때마다 평균 계산 후 UART2로 전송

- CubeMX 설정

- TIM3 : 1kHz 주기, Update interrupt, TRGO = Update 로 설정
- ADC1: Regular conv., External trigger = TIM3 TRGO, Sampling time = 28.5 cycles
- DMA: ADC1 → DMA1_channel1, circular, length = 100, byte/halfword = halfword



✓ main 수정

```
/* USER CODE BEGIN PV */
uint16_t adc_buf[ADCBUF_MAX];
/* USER CODE END PV */

/* Private function prototypes -----
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code -----
/* USER CODE BEGIN 0 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1){
        uint32_t sum = 0;
        for (int i=0;i<ADCBUF_MAX;i++) sum += adc_buf[i];
        float avg = (float)((sum / ADCBUF_MAX)/4096.0*3.3);
        char line[32];
        int len = sprintf(line, "ADCavg=%.2f\r\n", avg);
        HAL_UART_Transmit(&huart2, (uint8_t *)line, len, 50);
    }
}

/* USER CODE END 0 */
```

```
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_TIM3_Init();
    MX_USART2_UART_Init();
    MX_ADC1_Init();
    /* USER CODE BEGIN 2 */
    HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc_buf, ADCBUF_MAX);
    HAL_TIM_Base_Start(&htim3);
    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}
```

✓ main.h 수정

```
/* USER CODE BEGIN Private defines */  
#define ADCBUF_MAX 100  
/* USER CODE END Private defines */
```

✓ 이렇게 하면 TIM3가 1kHz → ADC1 트리거 → DMA 에 100개 샘플 저장 → 100개 채워질 때마다 DMA1_Channel1_IRQHandler ISR호출 → HAL_DMA_IRQHandler 호출 → HAL_ADC_ConvCpltCallback 함수 호출 → 이 함수 내에서 평균 계산 후 UART2로 전송하게 됨.

- adc_buf 값 Live Watch 통해서 확인
- avg 값 Tera Term 통해서 확인

✓ 과제 1: DMA + PWM

- TIM 의 PWM출력 duty 를 과제1과 같은 LUT이용해 미리 연속된 값 저장해두고
- DMA 통해서 TIM->CCR1의 값 받아서 연속해서 duty 점차 변화시키면 (DMA circular mode)
- LED의 밝기가 점차 밝게 또는 점차 어두워지도록 변하게 할 수 있음
- Timer의 PWM 주파수는 1kHz로 설정
- Breadboard 에 저항+LED 회로 구성해서 실험

✓ 과제 2: TIM3 → DMA trigger → DAC로 데이터 전송 (STM32F302 보드 사용)

- 데이터는 Sine 파, Triangle 파, Sawtooth 파 세가지를 각각 look-up-table (LUT)로 저장해둔 세가지 vector
- Tera Term 에서 s 누르면 DAC에서 sine 파 나오게, t 누르면 triangular wave, w 누르면 sawtooth 파형 나오도록 구현 (TIM3 의 주파수와 LUT의 개수에 따라서 출력 파형의 주파수 결정됨)
- 정해진 파형이 나오는 중에 Tera Term 에서 u 누르면 주파수 10% 씩 up, d 누르면 주파수 10%씩 down 되게 구현